

Exercícios: do relacional ao MongoDB

Pedidos, documentos incorporados e consultas com find()

1. executar o modelo relacional
2. consultar com SQL
3. escolher o agregado
4. incorporar subdocumentos e arrays
5. reproduzir consultas no MongoDB

Roteiro

observar
modelar
migrar
consultar
justificar

O desafio exige quatro decisões conscientes

- Executar e compreender a base relacional fornecida.
- Escolher o pedido como raiz do agregado.
- Transformar relações em subdocumentos e arrays.
- Reproduzir consultas SQL com find(), projeção, sort e \$elemMatch.

O objetivo não é converter tabelas: é modelar para as consultas.

Caso Loja Horizonte: o histórico do pedido precisa sobreviver às mudanças

- A loja mantém clientes, endereços, produtos, categorias, pedidos e itens.
- No relacional, JOINS recompõem o pedido completo.
- No MongoDB, o pedido deve ser lido como um agregado.
- Nome e preço do produto podem mudar após a compra.

Pergunta central: quais dados devem ser retratos históricos?

O modelo relacional distribui o pedido em seis tabelas

clientes

cadastro e contato

endereços

destinos do cliente

categorias

classificação do
produto

produtos

catálogo e preço
atual

pedidos

evento de compra

itens_pedido

produtos
comprados

1. executar

criação e carga

2. consultar

JOINS e filtros

3. modelar

incorporar ou
referenciar

4. migrar

documentos e
find()

Passo 1: crie e carregue a base PostgreSQL

```
-- psql
\i 01_loja_relacional.sql

-- confira a carga
SELECT * FROM pedidos ORDER BY id_pedido;
SELECT * FROM itens_pedido ORDER BY id_pedido;
```

- O script recria todas as tabelas e restrições.
- A carga contém 4 clientes, 7 produtos e 6 pedidos.
- Confira as contagens antes de iniciar os exercícios.

Exercício 1: descubra como o pedido é reconstruído

- Execute uma consulta com pedidos + clientes + endereços.
- Acrescente itens_pedido + produtos + categorias.
- Identifique quais colunas descrevem o evento da compra.
- Marque os valores que precisam permanecer históricos.
- Registre quantos JOINS foram necessários.

Entrega rápida: uma consulta que mostre o pedido completo.

Cada tabela assume um papel no documento de pedido

pedidos	documento raiz	um documento por compra
clientes	cliente	snapshot incorporado
endereços	enderecoEntrega	subdocumento incorporado
itens_pedido	itens[]	array incorporado
produtos	item do array	nome e preço históricos
categorias	categoria do item	valor incorporado

Exercício 2: incorpore, referencie ou duplique?

- Cliente: guardar apenas id ou também nome e e-mail?
- Endereço de entrega: referência ou retrato completo?
- Itens: documentos incorporados ou coleção separada?
- Produto: preço atual ou preço praticado no pedido?
- Categoria: id, nome ou ambos?

O documento-alvo reúne tudo o que muda com a compra

```
{
  _id: 1, status: "PAGO",
  cliente: { id: 1, nome: "Ana Souza" },
  enderecoEntrega: { cidade: "São Paulo", uf: "SP" },
  itens: [
    { sku: "LIV-001", categoria: "Livros",
      quantidade: 1, precoUnitario: NumberDecimal("89.90") }
  ]
}
```

- cliente e enderecoEntrega são subdocumentos.
- itens é um array de documentos.
- Os dados representam o momento da compra.

Exercício 3: construa a validação antes da carga

```
db.createCollection("pedidos", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: [ /* campos do pedido */ ],
    properties: {
      cliente: { /* object + required */ },
      enderecoEntrega: { /* object + required */ },
      itens: {
        bsonType: "array", minItems: 1,
        items: { /* object + properties */ }
      }
    }
  }
})
// Inclua enum para status e minimum para valores numéricos.
```

Consulta 1: filtro simples

```
// SQL
SELECT * FROM pedidos
WHERE status = 'PAGO';

// MongoDB
db.pedidos.find({
  /* complete */
})
```

- WHERE vira um objeto de filtro.
- O valor continua sendo uma string.
- Resultado esperado: pedidos 1 e 6.

Escreva a consulta antes de executar.

Consulta 2: projeção, ordenação e limite

```
// SQL
SELECT id_pedido, data_pedido, status
FROM pedidos
ORDER BY data_pedido DESC
LIMIT 3;

// MongoDB
db.pedidos.find(
```

- O segundo argumento de find() é a projeção.
- sort({campo:-1}) ordena de forma decrescente.
- Mantenha apenas _id, dataPedido e status.

Escreva a consulta antes de executar.

Consulta 3: filtro no subdocumento

```
// SQL
SELECT p.* FROM pedidos p
JOIN enderecos e ON
e.id_endereco=p.id_endereco_entrega
WHERE e.uf='SP';

// MongoDB
db.pedidos.find({
```

- O JOIN desaparece porque o endereço foi incorporado.
- Use enderecoEntrega.uf.
- Resultado esperado: pedidos 1, 2 e 3.

Escreva a consulta antes de executar.

Consulta 4: o JOIN vira acesso ao snapshot

```
// SQL
SELECT p.* FROM pedidos p
JOIN clientes c ON c.id_cliente=p.id_cliente
WHERE c.nome='Ana Souza';

// MongoDB
db.pedidos.find({
  /* campo dentro de cliente */
```

- O nome foi duplicado intencionalmente no pedido.
- A consulta não depende da coleção clientes.
- Resultado esperado: pedidos 1 e 3.

Escreva a consulta antes de executar.

Consulta 5: buscar valor dentro do array

```
// SQL
SELECT DISTINCT p.* ...
JOIN categorias c ...
WHERE c.nome='Livros';

// MongoDB
db.pedidos.find({
  /* caminho até categoria */
```

- A notação de ponto alcança campos de qualquer item.
- Use itens.categoria.
- Resultado esperado: pedidos 1, 4 e 6.

Escreva a consulta antes de executar.

Consulta 6: condições no mesmo item

```
// SQL
WHERE pr.sku='INF-002'
  AND i.quantidade >= 2;

// MongoDB
db.pedidos.find({
  itens: { $elemMatch: {
    /* duas condições */
```

- \$elemMatch mantém as condições no mesmo elemento.
- Use \$gte para quantidade mínima.
- Resultado esperado: apenas o pedido 2.

Escreva a consulta antes de executar.

Exercício 5: entregue as seis consultas nos dois modelos

- Execute `02_consultas_relacionais_exercicios.sql`.
- Escreva a versão MongoDB com `find()`.
- Compare a quantidade e os IDs retornados.
- Explique por que alguns JOINS desapareceram.
- Use `aggregate()` apenas no desafio bônus de totalização.

Exercício 6: prove que as restrições funcionam

```
// cada insert deve falhar por um motivo diferente
db.pedidos.insertOne({ status: "PAGO" })

db.pedidos.insertOne({
  _id: 90, status: "DESCONHECIDO", itens: []
})

db.pedidos.insertOne({
  _id: 91, status: "PAGO", itens: [{ quantidade: 0 }]
})
```

- Identifique a regra violada em cada documento.
- Registre a mensagem retornada pelo MongoDB.
- Crie um quarto teste para UF ou CEP inválido.

Desafio: atualize um item sem substituir o pedido inteiro

```
db.pedidos.updateOne(
  { _id: 2, "itens.sku": "INF-002" },
  { $set: {
    "itens.$.quantidade": 3
  }}
)
```

Use o operador posicional \$ para alterar o elemento encontrado no array.

Entrega da dupla: modelo, scripts e justificativas

- Diagrama relacional com cardinalidades.
- Documento de pedido com subdocumentos e array.
- Validator da coleção e carga de seis pedidos.
- Seis consultas SQL e seis consultas MongoDB.
- Quatro testes de documentos inválidos.

Inclua uma justificativa para cada incorporação ou referência.

Uma solução possível usa pedido como raiz do agregado

```
{
  _id, dataPedido, status, frete,
  cliente: { id, nome, email },
  enderecoEntrega: {
    logradouro, numero, cidade, uf, cep
  },
  itens: [{
    produtoId, sku, nome, categoria,
    quantidade, precoUnitario
  }]
}

// O pedido guarda snapshots históricos.
```

\$elemMatch evita combinar condições de itens diferentes

```
// SQL: SKU e quantidade pertencem à mesma  
linha  
WHERE pr.sku = 'INF-002'  
      AND i.quantidade >= 2  
  
// MongoDB: pertencem ao mesmo item do array  
db.pedidos.find({  
  itens: { $elemMatch: {
```

- Sem \$elemMatch, cada condição pode casar com um item diferente.
- Use notação de ponto para uma condição simples.
- Use \$elemMatch para condições correlacionadas.

O arquivo 05 contém o gabarito completo.

O exercício termina quando a modelagem explica as consultas

Modelo documental	30%	incorporação e snapshots coerentes
Validator e carga	20%	tipos, required, enum, arrays e objetos
Consultas equivalentes	30%	mesmos resultados nos dois modelos
Testes de erro	10%	restrições demonstradas
Justificativa	10%	decisões ligadas ao padrão de acesso
Pergunta final	o que o documento eliminou?	e qual duplicação ele criou?

O modelo documental é uma decisão sobre leitura, histórico e mudança

- O pedido virou um agregado porque é lido como uma unidade.
- Subdocumentos eliminaram JOINS para cliente e endereço.
- O array de itens preservou produtos, quantidades e preços da compra.
- A duplicação exige intenção: alguns dados são snapshots, não cópias acidentais.

Se a consulta ficou simples, explique qual responsabilidade foi transferida para a escrita.